Taller05-Patrones Creacionales

Grupo #2

Integrantes:
- Valeska Urrutia
- Leonardo Estrada
- James Santana
- Valentina Concha

Sección A

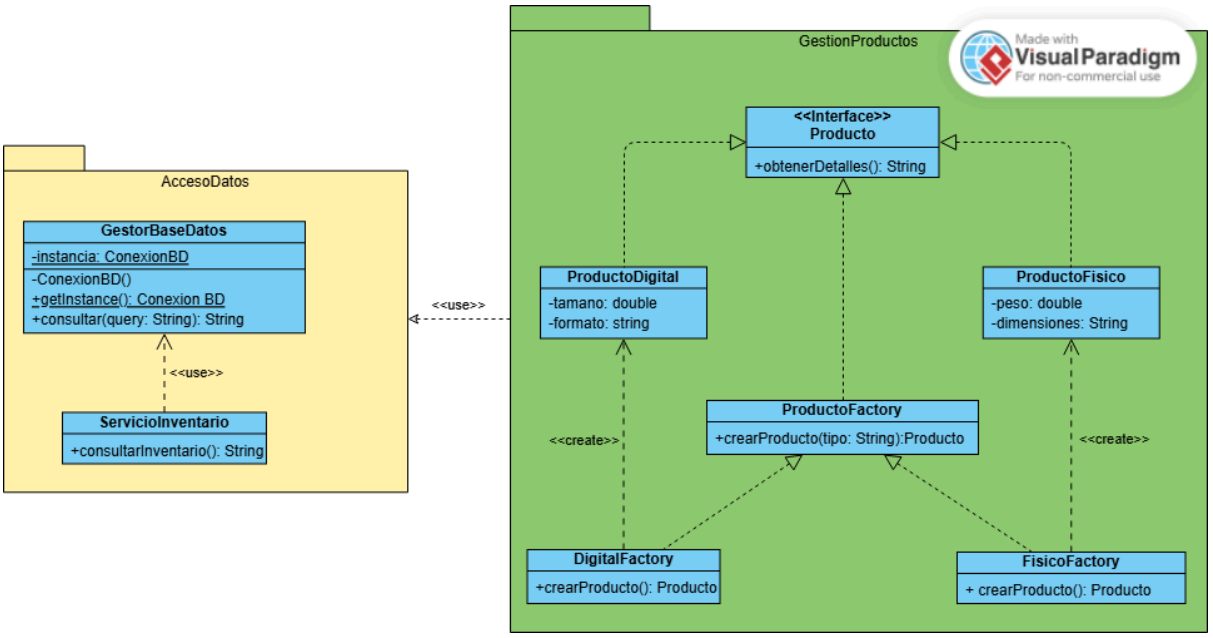
Escenario:
Considere el sistema para administrar los productos de una tienda que comercializa tanto artículos físicos como productos digitales. Permite registrar categorías, crear nuevos productos con sus atributos específicos, consultar información de inventario y precios, y almacenar toda la información en una base de datos central. El sistema busca mantener la coherencia de los datos y ofrecer una estructura flexible que permita añadir nuevos tipos de productos sin modificar el resto del código.
Para asegurar que todas las operaciones trabajen sobre la misma fuente de información, se define un componente encargado de gestionar la conexión con la base de datos y atender todas las consultas y actualizaciones. Dicho componente garantiza que solo exista una instancia activa en todo el sistema, evitando duplicación de recursos y posibles conflictos de acceso a los datos.
Por otro lado, cuando se necesita crear un producto, el sistema emplea un mecanismo que decide dinámicamente qué tipo de objeto construir según el tipo solicitado. Así, si el producto es físico, incluirá peso y dimensiones; si es digital, contendrá tamaño y formato. Este mecanismo facilita la extensión del sistema para soportar nuevos tipos de productos, manteniendo el código organizado y fácil de mantener.

Patrones:
1. Patrón Singleton
    a. Objetivo: la clase encargada de gestionar la conexión con la base de datos debe poseer una única instancia en todo el sistema, dando un un punto de acceso global a todo el sistema.
    b. Motivación: se necesita que todas las operaciones de la tienda trabajen sobre la misma fuente de información central. Si se tiene muchas instancias se llegaría a generar duplicaciones de recursos y conflictos de acceso a los datos.
    c. Consecuencias:
        i. Positiva: Se va a tener un código organizado y fácil de mantener
        ii. Negativa: Dificulta las pruebas unitarias al actuar de forma global
    d. Relación con Solid: No cumple con SRP porque la clase solo responde a la gestión de conexión, liberando al resto del sistema de esta tarea.
2. Patrón Factory Method

a. Objetivo: hacer que la clase creadora de las decisiones de instanciar un producto físico o un producto digital según lo que necesite el sistema.
b. Motivación: permitir la creación de productos con atributos distintos sin llenar el código de condicionales, facilitando la llegada de nuevos productos en el futuro.
c. Consecuencias:
    i. Positiva: permite añadir nuevos tipos de productos sin alterar el código existente
    ii. Negativa: incrementa la cantidad de clases e interfaces
d. Relación con Solid:
    i. Cumple con SRP porque separa la responsabilidad de construir los productos de la responsabilidad de gestionar la tienda.
    ii. Cumple con OCP porque el sistema queda abierto a nuevos tipos de productos y cerrado a la modificación de las clases que ya existen.

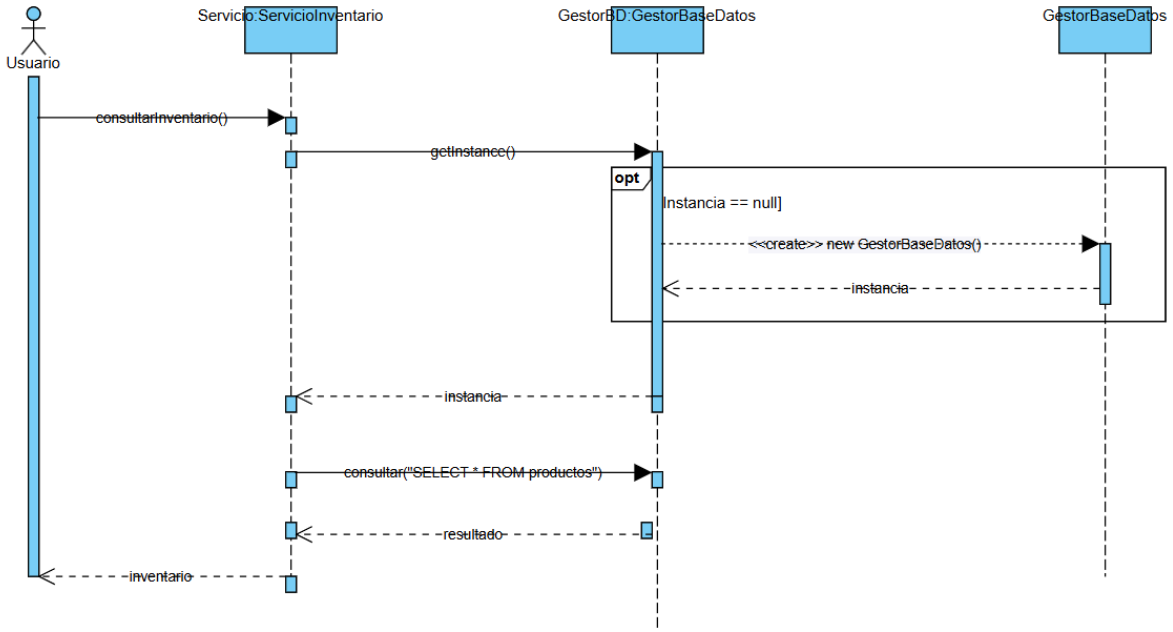
Sección B

1. Diagrama de clases

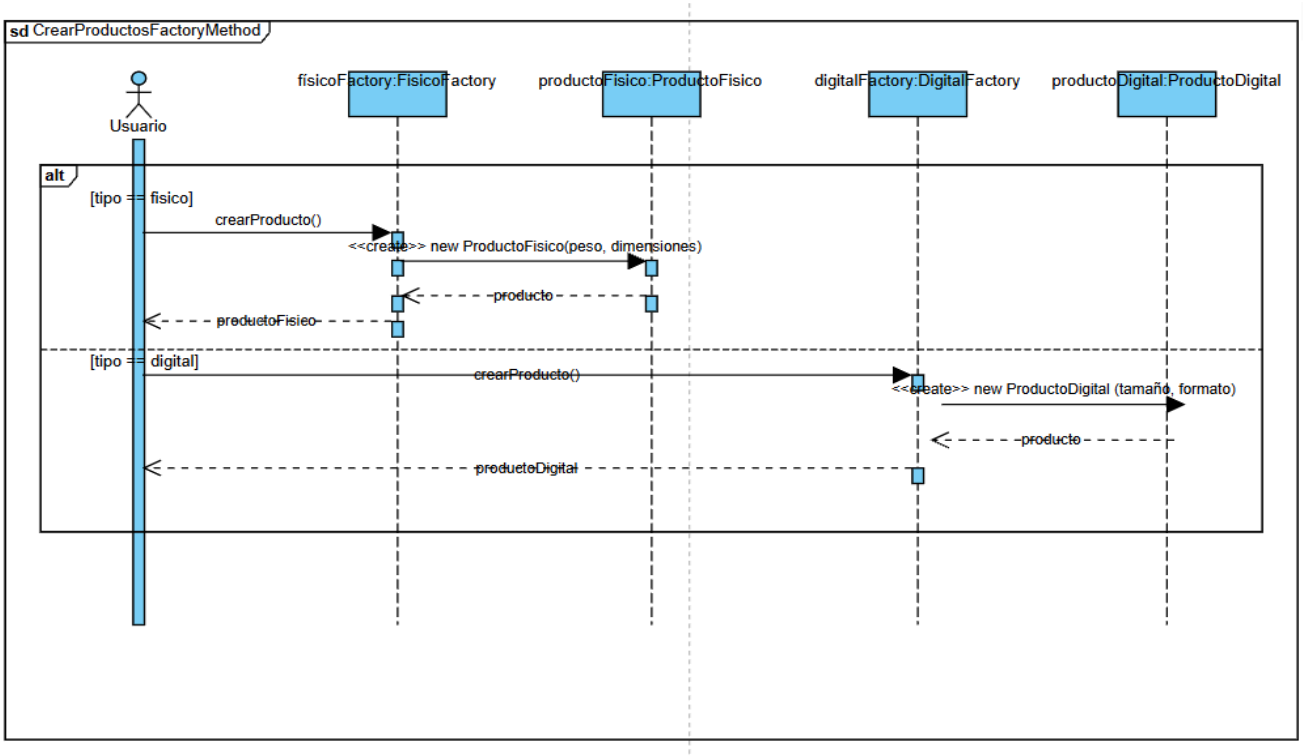


2. Diagramas de Secuencia

i. Singleton



2. Diagrama Factory



Sección C

https://github.com/JamesSantana911/Taller05-Patrones-Creacionales